Postman API testing guide

This document describes how to test the LinkedIn Profile Service in Postman. The service runs on your machine. LinkedIn session cookies stay in the server .env file; Postman does not send li_at. Never put cookies or tokens in this guide or in a shared collection.

1. Prerequisites

- Python venv activated in the project folder.
- .env saved with LINKEDIN_LI_AT and LINKEDIN_JSESSIONID (plus bcookie and lidc in LINKEDIN_EXTRA_COOKIES when LinkedIn returns 401).
- Server running: uvicorn app.main:app --host 127.0.0.1 --port 8000
- Postman desktop or web (desktop is easier for localhost).

Confirm the process is up: open http://127.0.0.1:8000 in a browser, or run the Health request below.

2. Postman setup (once)

Create a collection

- New -> Collection -> name it LinkedIn Profile Service.

Collection variable

- Collection -> Variables.
- Add base_url = http://127.0.0.1:8000 (current value and initial value).
- Use {{base_url}} in every request URL.

No auth tab on the collection

Do not add Bearer tokens or LinkedIn cookies in Postman. Authentication to LinkedIn happens inside the backend from .env. After you change .env, restart uvicorn before retesting.

3. Endpoints

Method	Path	Auth to this API	Purpose
GET	/	None	HTML test UI (not required in Postman)
GET	/health	None	Process is running
GET	/ready	None	True if .env session cookies loaded
POST	/profile	None (server uses .env)	Fetch and normalize a LinkedIn profile

4. Request: GET Health

Checks that uvicorn is listening. Does not call LinkedIn.

```
GET {{base_url}}/health
```

Postman: New request -> GET -> {{base_url}}/health -> Send. No body, no headers required.

Expected

```
HTTP 200
{
  "status": "ok"
}
```

If connection refused: the server is not running or the port is not 8000.

5. Request: GET Ready

Checks whether LINKEDIN_LI_AT and LINKEDIN_JSESSIONID were loaded at process start.

```
GET {{base_url}}/ready
```

Expected when .env is loaded and uvicorn was started after saving .env

```
HTTP 200
{
  "session_configured": true
}
```

Expected when cookies are missing

```
HTTP 200
{
  "session_configured": false
}
```

session_configured false with a saved .env usually means uvicorn was not restarted after the file change.

6. Request: POST Profile (primary test)

This is the challenge endpoint. Postman sends only a public LinkedIn profile URL.

```
POST {{base_url}}/profile
```

Headers

Header	Value
Content-Type	application/json

Body (raw JSON)

```
{
  "linkedin_url": "https://www.linkedin.com/in/username/"
}
```

Replace username with a real vanity slug. Query strings are allowed (?trk=...). Company pages and other hosts are rejected.

Postman clicks

- Method POST, URL {{base_url}}/profile
- Headers: Content-Type = application/json (Body -> raw -> JSON also sets this).
- Body -> raw -> JSON -> paste the object above.
- Send. First LinkedIn round-trip can take a few seconds.

Expected success (HTTP 200)

```
{
  "profile": {
    "name": "string",
    "headline": "string",
    "location": "string",
    "about": "string",
    "image_url": "string",
    "experience": [
      {
        "title": "string",
        "company": "string",
        "location": "string",
        "dates": "string",
        "description": "string"
      }
    ],
    "education": [
      {
        "school": "string",
        "degree": "string",
        "field": "string",
        "dates": "string"
      }
    ],
    "skills": ["string"],
    "certifications": [
      { "name": "string", "issuer": "string", "date": "string" }
    ],
    "languages": [
      { "name": "string", "proficiency": "string" }
```

```
    ]
  }
}
```

Empty strings and empty arrays are valid when LinkedIn did not return that section (certifications and languages are often empty). Tests should assert HTTP 200, a profile object, and name or headline populated for a known public profile.

7. Negative tests (Postman)

Case	Body / URL	HTTP	error.code
Invalid URL (company page)	{"linkedin_url":"https://www.linkedin.com/company/example/"}	400	invalid_linkedin_url
Malformed JSON / missing field	{}	422	(FastAPI validation; detail array, not error.code)
linkedin_url not a URL	{"linkedin_url":"not-a-url"}	422	(validation)
Session expired / LinkedIn 302	valid profile URL	401	linkedin_auth_failed
Profile cannot be resolved	valid shape, unknown slug	404	profile_not_found
LinkedIn 429	valid profile URL	429	linkedin_upstream_error
.env cookies missing at startup	valid profile URL	500	missing_credentials
LinkedIn 999 / other failure	valid profile URL	502	linkedin_upstream_error

Error envelope (except FastAPI 422):

```
{
  "error": {
    "code": "linkedin_auth_failed",
    "message": "human readable text"
  }
}
```

8. Tests tab (optional Postman scripts)

On POST /profile, Tests tab example:

```
pm.test('status is 200', function () {
  pm.response.to.have.status(200);
});
pm.test('profile has name', function () {
  const json = pm.response.json();
  pm.expect(json.profile).to.be.an('object');
  pm.expect(json.profile.name).to.be.a('string');
  pm.expect(json.profile.experience).to.be.an('array');
  pm.expect(json.profile.education).to.be.an('array');
  pm.expect(json.profile.skills).to.be.an('array');
});
```

On GET /health:

```
pm.test('healthy', function () {
  pm.response.to.have.status(200);
  pm.expect(pm.response.json().status).to.eql('ok');
});
```

9. Collection runner

- Save Health, Ready, Profile (success), Profile (invalid URL) as four requests.
- Runner -> select those requests -> keep delay 0 for Health/Ready.
- Do not loop POST /profile at high volume. LinkedIn will invalidate the session (401).
- One live profile lookup per run is enough for a demo.

10. Import as cURL (alternative to clicking)

Postman: Import -> Raw text -> paste:

```
curl --request GET 'http://127.0.0.1:8000/health'
```

```
curl --request GET 'http://127.0.0.1:8000/ready'
```

```
curl --request POST 'http://127.0.0.1:8000/profile' \
  --header 'Content-Type: application/json' \
  --data '{"linkedin_url":"https://www.linkedin.com/in/username/"}'
```

Then replace username and save into the collection.

11. Troubleshooting

Symptom	What to do
Could not send request / ECONNREFUSED	Start uvicorn on 127.0.0.1:8000. In Postman web, localhost may be blocked; use the desktop app.
304 on browser static files; Postman still old?	Postman does not cache this API. Unrelated. Hit Send again.
session_configured false	Save .env, stop uvicorn (Ctrl+C), start it again.
401 linkedin_auth_failed	Refresh li_at, JSESSIONID, bcookie, lidc from Chrome; save .env; restart uvicorn.
422 on POST /profile	Body must be raw JSON with linkedin_url as a full https URL, not form-data.
200 but empty name and empty lists	Unusual; treat as a failed parse. Retry once. If it persists, the LinkedIn payload shape changed.

12. Security notes for testers

- Do not put LINKEDIN_LI_AT in Postman environment variables or screenshots.
- Do not export a collection that contains real profile PII to a public gist.
- Do not commit .env.

Document generated for local Postman testing of this repository. Interactive OpenAPI UI is also at http://127.0.0.1:8000/docs